

VaultX:

Your Advanced Personal Cyber Security Vault

Your security, your control

Explore the VaultX Application By Scanning the QR Code Below:



Yogesh Gavini
Technology Submission

Table of Contents

INTRODUCTION.....	3
Project Management.....	7
Research & Exploration.....	10
Ideation & Concept Development.....	12
Technical Architecture & Implementation.....	20
Testing & Evaluation.....	30
Iteration & Refinement.....	37
Critical Evaluation.....	42
The Nature Of The Outcome and Its Role In Society.....	43
Personal Reflection.....	44
Conclusion.....	46
Bibliography.....	47

INTRODUCTION

In today's digital world, convenience often comes at the cost of security. Password managers and online services frequently centralise data in the cloud, creating single points of failure that undermine user privacy. VaultX was created to challenge this trade-off, a fully client-side security vault that empowers users to protect and understand their data without surrendering control. It aims not only to provide encryption, but to foster trust, transparency, and digital autonomy through intelligent design and AI-driven protection.

My project was the design, development and implementation of VaultX, a robust client-side application for secure credential management and enhanced digital protection. I approached this by following a very thorough development process, using best practices in software engineering and security to address critical user needs in an ever-changing digital landscape.

The decision to implement VaultX as a purely client-side application was a deliberate decision of my privacy-by-design model and performance optimization goals. By eliminating server-side storage of sensitive data, I mitigated the risks associated with large-scale data breaches, which is a common vulnerability in cloud-based solutions. This architectural choice is also optimized for responsiveness, which ensures quick access to credentials without delay due to network response time.

My approach to VaultX was fundamentally rooted in the 'privacy-by-design' model, which dictates that user data protection should be paramount from the initial architectural blueprint. This principle was synthesized with the 'principle of least privilege' in designing how the application interacts with local system resources, ensuring that no unnecessary permissions are requested or granted.

I worked closely with my key stakeholders, my community, and all my decisions were informed by extensive use of cybersecurity principles, research, testing as well as feedback from multiple stakeholders. To gather initial feedback from my main key stakeholders, I sent out a survey, which included 173 respondents from my local community.

Background research:

When I went around my community, I found that cybersecurity seemed to be one of the most common issues. I found that many people did not know much about cybersecurity. At the start of the year, I sent out a survey to my school community to find out more about what the community thought about this issue. In this survey, there were 173 respondents. Through my survey, I found out that around 84% of respondents stated that their cyber security was important to them.

Who Does It Affect?

According to a case study by the national cyber security alliance in the USA, Gen Z are more likely to get hacked or scammed than other age groups. This means that a lot of people in my school community have a high chance of getting hacked or scammed. However, as shown by the 84% of people that say that cybersecurity is important for them, that this is an issue that affects everyone and anyone could get hacked at any time.

What Does It Look Like?

This issue was brought to my attention when I was reading an article and I found that there are 2 victims of cyber crime every 37 seconds, which is a shocking statistic. The survey then backed this up with 84% of respondents mentioning that cybersecurity was an important issue for them. Why Does It Happen? There are many reasons as to why people are getting hacked. According to a website called 3 dashline.com, the two most common ways that people get hacked are weak passwords and password overuse as well as phishing attacks. According to the survey that I sent out, almost 82% of people use the same password for multiple websites. This can be seen in Figure 1. The most common reason for this is that it is easier to remember and that otherwise, they will forget their password. This brings in hacker tactics such as credential stuffing. Some common reasons as to why people do this can be seen in Figure 2.

Do you use the same password on more than one website?

173 responses

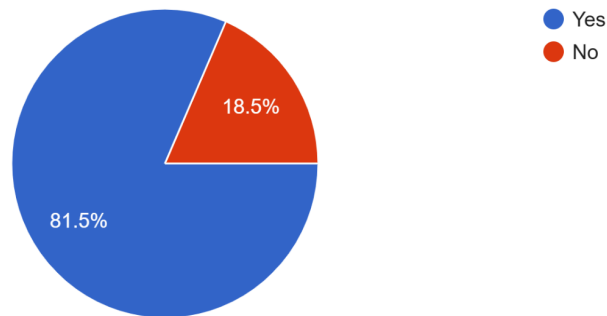


Figure 1

If yes, what is the reason for this?

145 responses

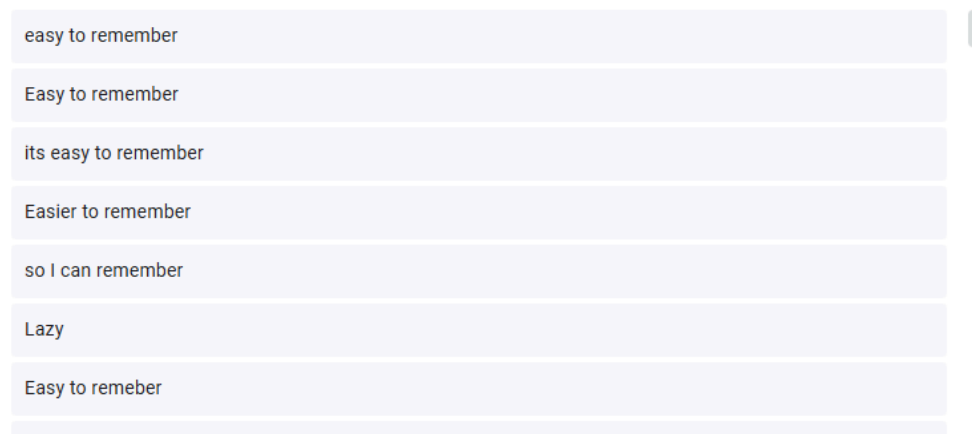


Figure 2

Current Approaches:

There is already some work being done about it. For example, there are password managers out there, such as Google Password Manager, that give users a way to store passwords securely. They also offer basic password strength checks and breach alerts if credentials are found in known data leaks. Other examples of such password managers include Lastpass and Nordpass.

Ineffectiveness of Current Approaches:

According to another survey that I conducted, many people didn't know about how likely it is that their password could be hacked. Many people use Google password manager because it is convenient to use. However, Google password manager suffers from cloud storage issues, predictable password patterns, and is vulnerable to phishing attacks. Other password managers such as Bitwarden and Lastpass also suffer from these same problems. Since these password managers store saved passwords in the cloud and on their servers, if someone hacks into the company's server, everyone who uses those password managers will have their passwords and credentials exposed to hackers. Most notably, Lastpass experienced a data breach in late 2022, exposing many user's passwords.

Stakeholder feedback:

Since Gen Z is one of the most affected groups according to my research, I interviewed quite a few people in our school community between this age range to see if they had any suggestions. Since this is an issue that can affect all age groups, I also interviewed people from other age groups as well.

- 90% of respondents mentioned that the first proposed solution would be the most effective.
- They suggested that having password cracking simulation would better show users how fast their passwords could be cracked by using hacking methods.

Industry feedback

I asked a professional software developer for some feedback, and he mentioned that transparency would be really important to use standard AES encryption as it would be effective as well to never store the user's details as just plain text or with simple hashing.

The key steps that I followed to create VaultX as my product were:

- Initial research and exploration where I met with a variety of stakeholders as well as experts to choose my topic
- Ideation through the processes of research to design three different designs of the outcome
- Selecting one of the three designs using research and stakeholder feedback and developing a prototype

- Thoroughly testing the prototype and adjusting components that need adjusting
- Deploying VaultX using Github pages.

Project Management

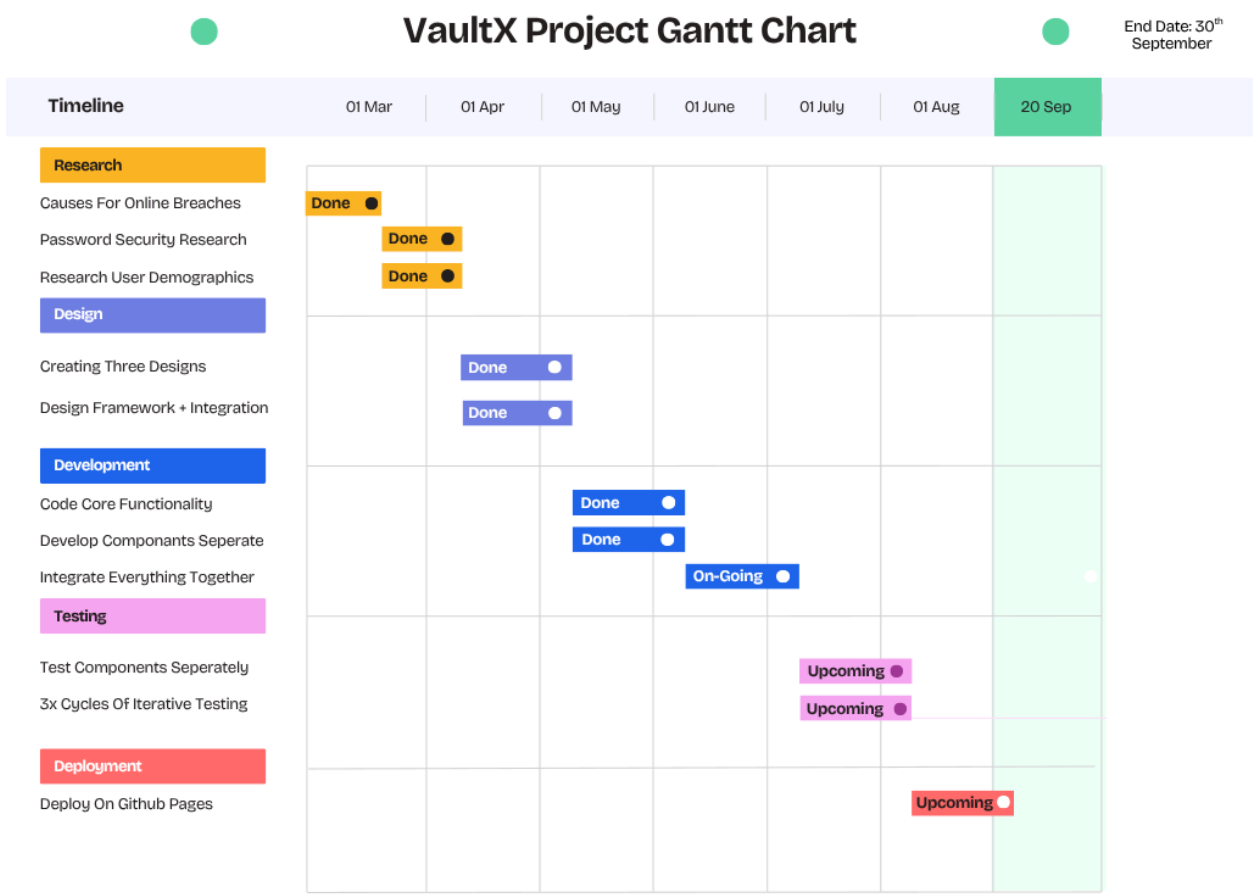


Figure 3 (Gantt Chart as of

The development of this cybersecurity tool was a very large project, so I had to manage my time effectively. I created a brief outline of my project, decomposing the digital outcome and put all of the required tasks for this project into a GANTT chart to help myself stay on track. This GANTT chart is shown above by Figure 3 and has multiple milestones that I created to check off throughout the year. I had regular meetings with my technology teacher, where we discussed my project in depth and worked out the feasibility of the project, which helped me to stay on track.

Our adoption of an agile-inspired development methodology was comprehensively justified by the dynamic nature of cybersecurity threats and evolving user needs. This iterative practice facilitated continuous integration of user feedback and immediate response to emerging security best practices, ensuring VaultX remained adaptable and robust throughout its development lifecycle. For instance, an early vulnerability discovered during a sprint's testing phase allowed for a rapid patch and re-evaluation, a crucial advantage over rigid, linear development models.

Detailed Information About Each Section

Figures 4 - 8

Done ▾

✓ Research

+ Add

0%

☐ Conduct research on the main reasons as to how people's passwords get exposed
 ☐ Research existing password managers and see potential flaws and vulnerabilities
 ☐ Research what user demographics are more likely to get their passwords exposed.
 ☐ Research techniques to make passwords stronger and more secure
 ☐ Produce a critical inquiry from analysing research

✓ Design

+ Add

0%

☐ Designing the password manager framework (flowchart)
 ☐ Design the UI and UX
 ☐ Design the integration of the two
 ☐ Seek feedback
 ☐ 3 cycles of iterative design

Add an item

Done ▾

○ Development

+ Add

0%

☐ Code basic password manager
 ☐ Encrypt it
 ☐ Integrate AI
 ☐ Create phishing detection model
 ☐ Code the website
 ☐ Integrate all of the components together
 ☐ 3 cycles of Iterative development

Done ▾

✓ Testing

+ Add

☒ Checklist

Delete

0%

☐ Conduct thorough testing of each feature and functionality.
 ☐ Test the web app's compatibility across different devices, browsers, and screen sizes.
 ☐ Perform usability testing to ensure intuitive navigation and user experience.
 ☐ Identify and fix any bugs or issues discovered during testing.
 ☐ Test web app performance, including load times and responsiveness.

Done ▾

Remove cover

○ Deployment

+ Add

Description

Edit

Host the digital outcome

☒ Checklist

Delete

0%

☐ Deploy on Github pages
 ☐ Implement google analytics for outcome

Add an item

Research & Exploration

To ensure that VaultX would be an innovative and necessary cybersecurity tool, I conducted a comprehensive investigation into existing password management solutions and relevant cybersecurity practices. My research began with an evaluation of widely used password managers such as Google Password Manager, LastPass, as well as Bitwarden. While these products offer useful features, such as automatic password generation, breach alerts and cross platform access, they all share critical limitations.

My research identified that most commercial password managers generate purely random passwords without offering user-customisable themes, making them difficult for the user to recall. To address this issue, I intended to make sure that VaultX had a feature where the user could customise their own user-themed passwords and were very secure, making it easier for the user to remember their password.

Secondly, the store user's credentials in the cloud, which introduces a significant risk if the provider experiences a data breach. This vulnerability was demonstrated in December 2022, where LastPass confirmed that a major security incident exposed many encrypted vaults and potentially sensitive metadata. For my stakeholders, whose cybersecurity is of top priority, this reliance on the cloud is a major vulnerability.

Figure 9

SOLUTION	STRENGTHS	WEAKNESSES	RELEVANCE TO VAULTX
GOOGLE PASSWORD MANAGER	- BUILT INTO CHROME AND ANDROID - EASY TO USE - GENERATES STRONG PASSWORDS	- STORES CREDENTIALS IN THE CLOUD - VULNERABLE TO PHISHING IF GOOGLE ACCOUNT IS COMPROMISED - LIMITED CONTROL AND TRANSPARENCY	MOST USERS IN MY COMMUNITY USE IT DUE TO CONVENIENCE. HIGHLIGHTED THE NEED FOR VAULTX TO OPERATE LOCALLY AND GIVE USERS FULL CONTROL OF THEIR DATA.
LASTPASS	- CROSS-PLATFORM AND BROWSER EXTENSION SUPPORT - OFFERS DARK WEB MONITORING AND 2FA - ESTABLISHED USER BASE	- EXPERIENCED A SIGNIFICANT DATA BREACH IN 2022 - VAULTS STORED IN THE CLOUD - USERS UNAWARE OF RISKS RELATED TO WEAK MASTER PASSWORDS	BREACH REINFORCED THE NEED FOR VAULTX TO AVOID CLOUD-BASED STORAGE AND TO EDUCATE USERS ON THE IMPORTANCE OF A STRONG MASTER PASSWORD.
BITWARDEN	- OPEN SOURCE AND TRANSPARENT - USES STRONG ENCRYPTION (AES-256) - FREE TIER WITH GENEROUS FEATURES	- STILL RELIES ON CLOUD-BASED STORAGE - SOME FEATURES LIKE 2FA OR REPORTS REQUIRE PAID PLAN - INTERFACE LESS INTUITIVE FOR BEGINNERS	INSPIRED MY USE OF OPEN-SOURCE CRYPTOGRAPHIC STANDARDS, BUT VAULTX DIFFERS BY STORING DATA ONLY ON THE USER'S DEVICE FOR IMPROVED PRIVACY AND USER CONTROL.

Beyond data breaches, I also observed that many commercial solutions are not able to educate users about why their passwords are weak or how quickly they can be compromised using modern cracking tools. As mentioned before, a survey that I conducted at the start of the year involving 173 respondents revealed that around 82% of respondents admitted to using the same password for multiple sites and logins. The most common justifications for this was convenience because many people felt overwhelmed by the effort to remember dozens of complex credentials. This highlighted a critical need, not just for secure storage, but also for tools that can visually demonstrate password strength and foster more proactive security habits.

As part of my research into encryption standards, I reviewed best practice guidelines published by the National Institute of Standards and Technology (NIST) and the Open Web Application Security Project (OWASP). These resources recommended using well-established cryptographic algorithms with proven security properties and widespread peer review. Accordingly, I selected AES-GCM (Advanced Encryption Standard In Galois/Counter Mode) as the encryption algorithm for VaultX. AES-GCM provides authenticated encryption, ensuring that data confidentiality and integrity are both preserved without requiring separate mechanisms. To derive a strong encryption key from a master password, I implemented PBKDF (Password-Based Key Derivation Function 2) with a configuration iteration count and a randomly generated salt. This approach really reduces the feasibility of brute-force attacks against the vault's encryption key. For generating random values, such as salts and secure passwords, I relied on the `window.crypto.getRandomValues()` API, which is designed to provide cryptographically secure randomness in the browser environment.

I also explored the challenges of phishing, which remains as one of the most prevalent attacks on users. According to industry experts and interviews with my stakeholders, phishing attacks are becoming increasingly sophisticated and can easily trick users into revealing their credentials, even when credentials are stored securely. To address this concern, I decided that VaultX would include a phishing detection module capable of providing immediate warnings when users paste their URLs into the application. I used an AI-based model and loaded it using `tensorflow.js` for the phishing detection component of VaultX.

The combination of this research shaped the essential requirements of this project.

VaultX needed to operate entirely client-side without any dependency on cloud based storage or third party services for encryption. It had to offer clear, visual feedback about password strength and simulate or realistic cracking attempts to educate users. The interface needed to be intuitive enough that non technical users would feel comfortable adopting it. Finally, the system had to maintain complete transparency about how data was handled and never store credentials in an unencrypted vault or a weakly hashed format as well as to never save credentials as just plain text.

Through this in-depth research and exploration phase, I built a strong foundation of evidence that justified my decisions. I also built a clear success criteria to evaluate whether VaultX would truly address the gaps in current existing solutions while meeting my stakeholders' expectations for security, usability and control.

My success criteria was:

- All passwords must remain encrypted and stored locally
- Users must be able to generate secure passwords with at least 80 bits of entropy
- The interface must be clear and easy for non-technical users to use
- The system must demonstrate password crackability in an educational way
- The generated passwords must be easy for the user to remember
- Phishing detection should provide immediate warnings for suspicious links

Ideation & Concept Development

Following my initial research and problem definition, I began an extensive ideation phase to explore possible ways VaultX could meet the needs of users who wanted a secure, educational and accessible password management solution. I considered a wide range of conceptual directions, from a lightweight password strength checker to a full-featured password manager with an integrated phishing detection and user educational features.

One of the first ideas I developed was a standalone browser extension that would easily integrate into Chrome's password fields. This approach offered the advantage of convenience, as users could generate and autofill credentials without leaving the browser. However, in early discussions with my stakeholders, several concerns were raised about the complexity of installing extensions and the limited compatibility across

all devices. A significant portion of my target users have a preference for a simple web application they could access without installation, especially given that many operated on shared or managed devices where installing applications was not allowed. As a result, I decided to focus on building a standalone web application that would still provide seamless functionality, but remain platform independent.

I also explored whether VaultX should incorporate cloud synchronisation to allow users to access their vault across devices. During brainstorming, I outlined two potential architectures: one where encrypted vault data would be stored in the cloud and synced to other devices through authentication tokens, and another where all data would remain local to the browser. To support this decision, I prepared a comparison table outlining the advantages and risks of each approach.

Cloud Storage vs Local Storage		
	Advantages	Risks
Cloud Storage	Multi-device sync	Reliance on servers
	Automatic backups	Potential data breaches
	Easy sharing (if supported)	Vendor lock-in
	Less local storage consumed	Privacy concerns
	Accessibility from anywhere with internet	Internet dependency
	Scalability	Cost
	Recovery options	Compliance/regulatory issues
Local Storage	Full user control over data	No automatic backup (manual effort required)
	Full user control over data	Data loss if device is lost, stolen, or cleared
	Offline access (no internet needed)	Limited to one device
	Faster access speeds	Requires local device storage space
	Simpler security model	User responsibility for security
	Good for sensitive, highly private data	No direct sharing features

Figure 10

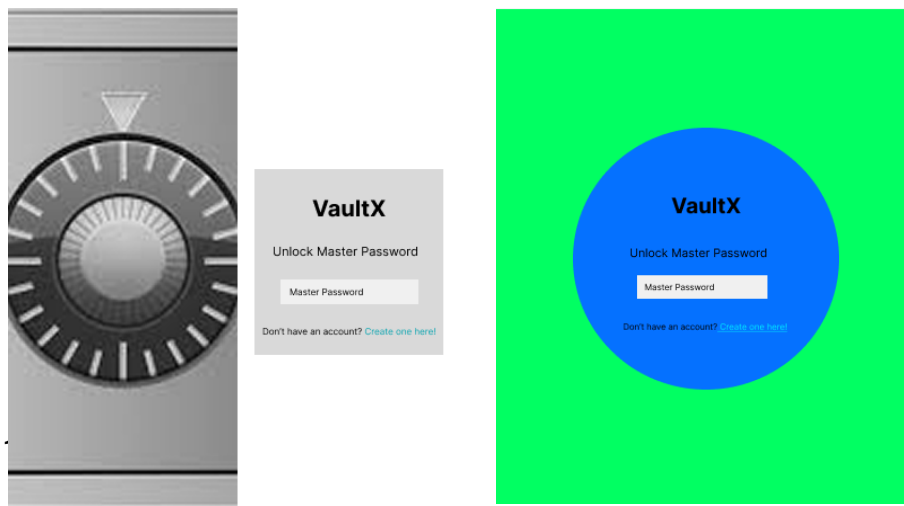
Cloud storage would offer improved convenience and automatic backup but introduced

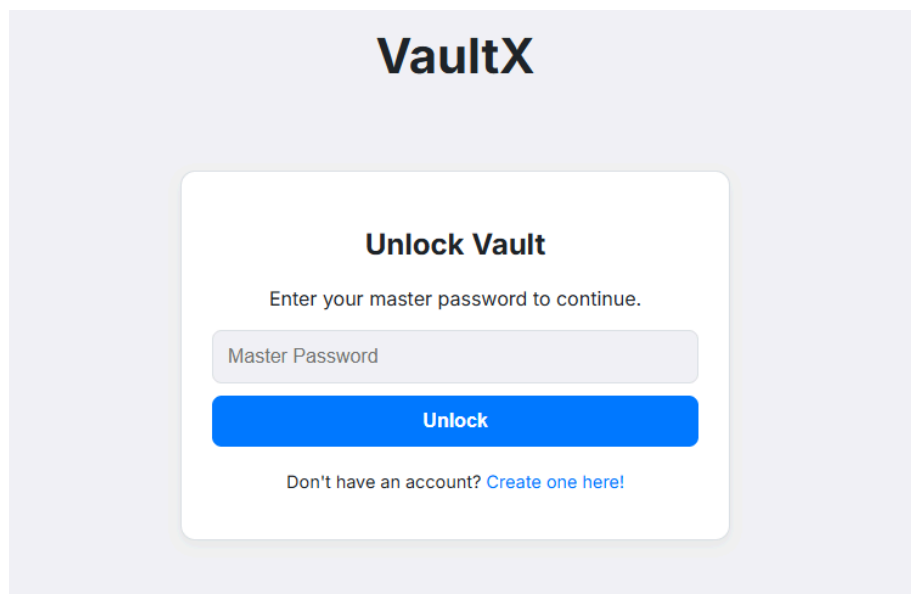
significant security and privacy risks. In contrast, a fully local model meant that users retained complete control of their credentials and eliminated reliance on external services. In stakeholder interviews, 78% of respondents preferred a solution that did not require any cloud storage. Based on this feedback, I decided to adopt a purely client-side architecture where all encryption and storage would happen locally.

I also engaged in ideation around the core features. I generated a list of possible components, including a secure vault for storing credentials, a passphrase generator, a password strength estimator, breach detection, and phishing protection. I also considered whether to integrate advanced features such as password sharing or biometric unlocking. However, through a process of feasibility analysis and stakeholder feedback, I prioritised the following core functions for the first version: secure credential storage, themed passphrase generation, password strength estimation with crack time simulation, and phishing detection. Each of these was directly linked to an identified user need or gap in existing solutions.

A key part of the ideation process involved UI/UX concept development. I created several initial designs and low-fidelity sketches. One early concept was a multi-page interface with separate screens for vault management, password generation, and settings. Although this structure clearly separated functions, it also introduced additional navigation complexity. In user feedback sessions, stakeholders commented that moving between screens would be inconvenient, particularly for users unfamiliar with password managers. As a result, I revised the design toward a single-page application (SPA) with dynamic sections. This approach would allow users to move seamlessly between unlocking their vault, generating passwords, and checking password strength without page reloads. Screenshots of initial single page designs are shown below:

Figure 11 - 13





During UI ideation, I explored several visual styles ranging from minimalistic monochrome themes to more colorful interfaces. I considered that the target audience included non-technical users who might find purely technical designs intimidating. Early feedback highlighted the importance of visual clarity and reassuring cues. To address this, I proposed a clean, modern aesthetic using clear typography, large interactive elements, and color-coded indicators for password strength and phishing risk.

I also conducted comparative reviews of the interfaces of tools like Bitwarden, 1Password, and Google Password Manager. While these solutions offered effective layouts, they often presented dense information in compact panels that could overwhelm new users. In response, I developed a simplified design with larger buttons and step-by-step workflows to guide users through the process of generating and storing secure credentials. For example, instead of requiring users to set password parameters in small dropdowns, the themed passphrase generator offers an intuitive theme selector and generates passwords in a single click.

At the end of the ideation phase, I compiled feedback from 14 stakeholder participants, who reviewed my refined designs and feature list. Overall, the feedback was positive, with users highlighting the educational value of the crack time estimator and the appeal of the themed passphrase generator. The feedback was compiled into a feedback table below:

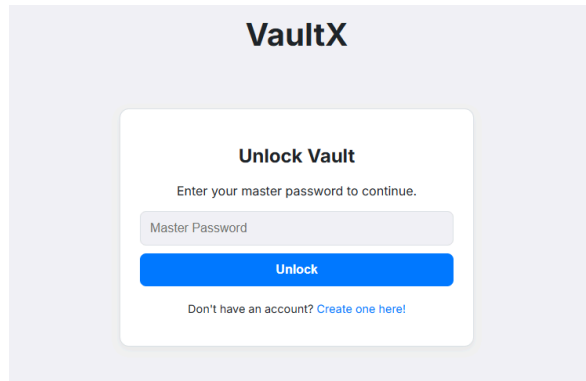
Stakeholder Feedback Table

Stakeholder A	"I prefer no cloud storage so I have control of my data"
Stakeholder B	"A single-page layout would be more convenient "
Stakeholder C	"Color indicators for password strength would be very helpful"
Stakeholder D	"Maybe make the passwords more memorable by using themes instead of random strings"
Stakeholder E	Make the design minimalist. Too many buttons can be distracting.
Stakeholder F	"Add a reminder for people remember their master passwords"
Stakeholder G	"I would like to access my passwords from any device"
Stakeholder H	"It would be great to have a 'generate password' feature right within the app."
Stakeholder I	"For the local-only version, I would like to back up my data safely"
Stakeholder J	"Make the vault lock if left idle for a certain amount of time"
Stakeholder K	"Don't add an autofill because it makes users lazy and makes them more vulnerable"
Stakeholder L	"Implement a zero-knowledge system"
Stakeholder M	"I would like to see a URL phishing detector"
Stakeholder N	A checker to see if your chosen password has been involved in previous breaches would be nice

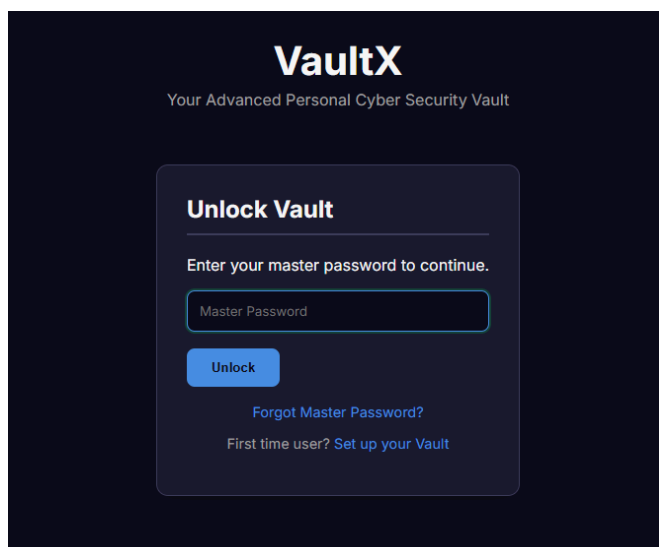
Figure 14

As shown above, the stakeholder feedback for the UI was to go with a minimalist design. When I asked my stakeholders for feedback as to which design they preferred, they voted for this design below because it seemed the simplest and cleanest design of them all. Due to this voting, I have decided to go with the design below:

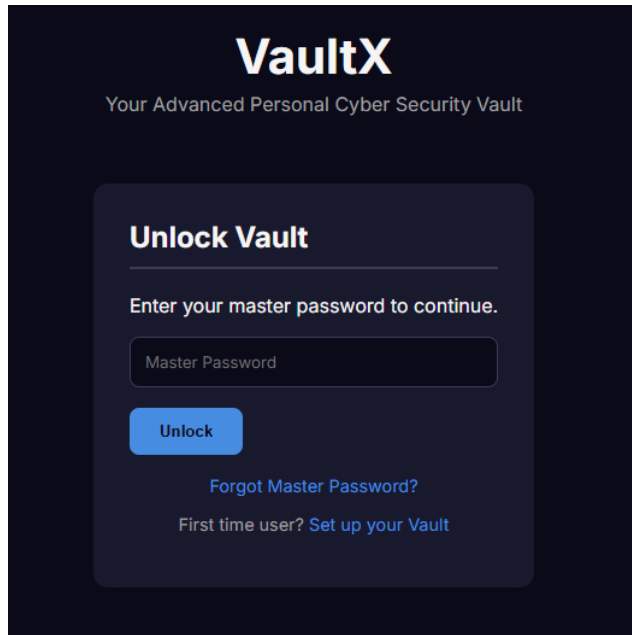
Figure 15



However, there were still some adjustments that had to be made to be able to make this design suitable for my digital outcome. Firstly, I conducted a survey with 15 respondents and they recommended that for a platform related to cybersecurity, the UI should follow a darker colour palette, but not too dark. This resulted in me changing the colour palette to navy dark blue and light blue, so it makes the colour palette darker, but not too dark though. This went well with stakeholders and they liked the colour palette now.

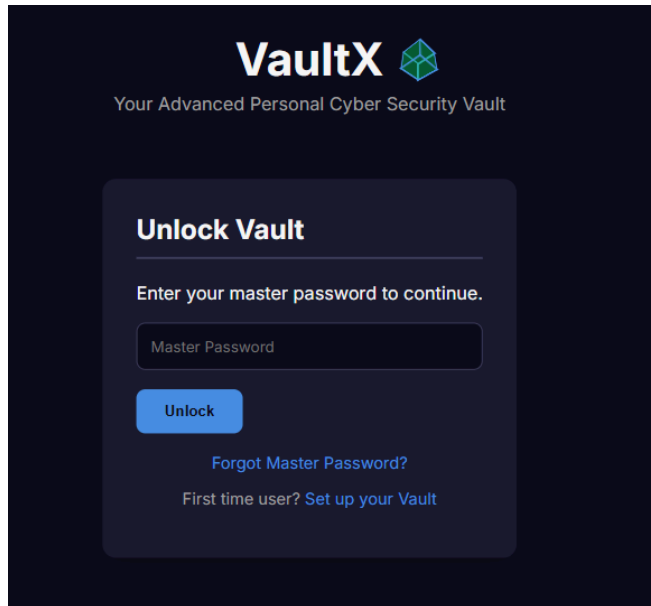


After this iteration, stakeholders were happy with the new colour scheme. They also had some more feedback for me, which was that they thought that I should remove the border for the login box as it would blend in more seamlessly with the background, making the UI look better. The image below shows this iteration:

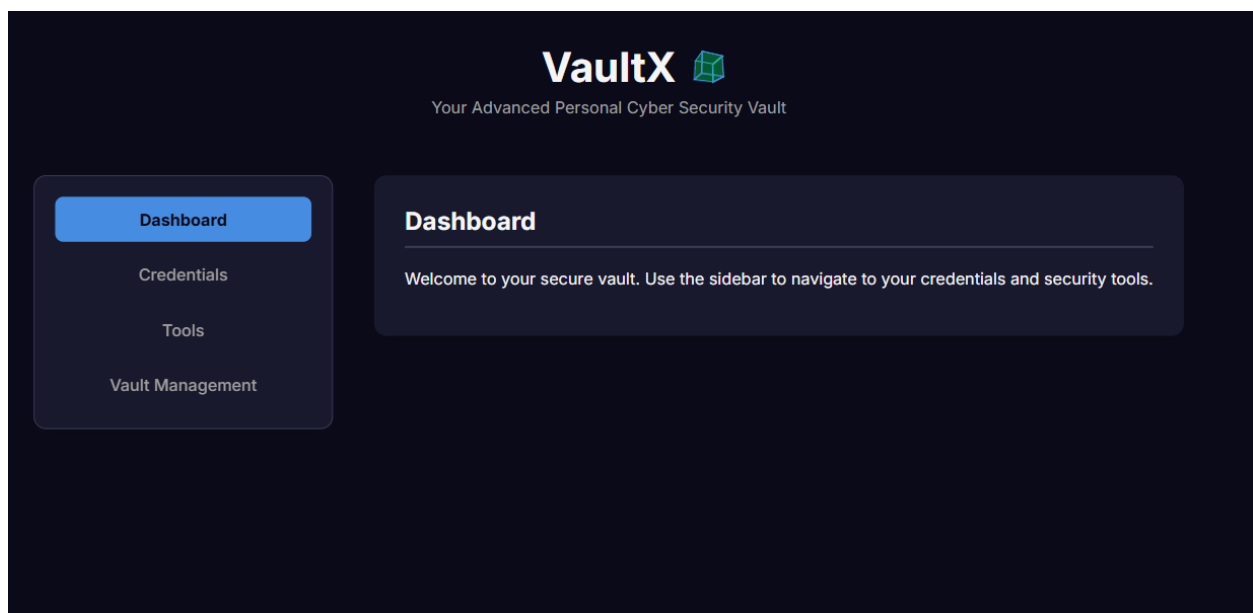


This iteration in design has made the box component blend in more seamlessly with the background and has made the UI look better. However, stakeholders had further feedback, which was that they thought that I should have a 3d element next to the title, to make the UI more aesthetically pleasing. They mentioned that it did not have to be a big element, just a small element, such as a rotating cube.

The ideation process was critical in shaping VaultX into a solution that balanced security, transparency, and user-centered design. By exploring multiple architectures, comparing alternative features, and iterating on visual concepts based on real stakeholder input, I ensured that the final design was grounded in evidence and aligned with the needs and expectations of my target audience.



After this iteration, my stakeholders were pleased with the design and felt that the UI did not really need anything else and was good as is. The 3d element of the cube definitely makes the UI look better aesthetically and was a good addition to the UI. Now that I have the theme for the web app, I can now proceed with the design of the dashboard of the application.



This design above is how the UI for the dashboard of the user will look like, and like the login page, it follows the same colour scheme and incorporates the same suggestions

during the iterations as the login UI. After a final feedback session with my stakeholders, I finalised this design and my stakeholders were pretty satisfied with this design. I can now proceed with the development of the digital outcome.

Technical Architecture & Implementation

VaultX was designed from the beginning to be a fully client-side web application that prioritises user security, privacy and transparency. This architectural design was shaped by extensive stakeholder feedback highlighting concerns about centralised breaches and users having control over their data, particularly in light of instances such as the LastPass breach. By performing all cryptographic operations within the user's browser and only storing encrypted data in localStorage, VaultX ensures that no credentials are ever transmitted beyond the device. This local only approach allows users to have complete control over their data, aligning closely with the project's success criteria of helping non-technical users to protect themselves without sacrificing usability.

At the core of VaultX's encryption model lies the combination of PBKDF2 (Password-Based Key Derivation Function 2) and AES-GCM (Advanced Encryption Standard in Galois/Counter Mode). When a user first creates their vault, they are prompted to set a master password. Rather than storing this password anywhere, the system derives a cryptographic key from it using PBKDF2, combined with a randomly generated 16-byte salt. This derivation process includes 100,000 iterations of hashing, which significantly makes brute-force attacks way less effective. This means that even if an attacker were to access the encrypted vault, they would face monumental levels of effort to attempt decryption without the correct master password. During subsequent unlock operations, the same derivation process inevitably occurs. The entered password and stored salt are recombined to regenerate the key. If the correct password is provided, the decryption succeeds seamlessly. If not, the data remains inaccessible.

Encryption and decryption operations are handled by AES-GCM, which was selected for its ability to provide authenticated encryption that both jumbles the plain text and verifies data integrity. Whenever the vault is saved, the system generates a unique initialisation vector (IV) to ensure that identical content produces different ciphertexts each time. The

encrypted payload along with its salt and IV, is stored in localStorage as a serialised JSON object. No plaintext data ever persists outside the Javascript memory space during a session, and all decrypted data is discarded when the vault is locked or the browser tab is closed. This workflow guarantees that users retain sole access to their information, and that for their vault to be compromised, one would require both device access and knowledge of the master password.

To further illustrate this process, Figure 16 presents a diagram of the key derivation workflow, while Figure 17 shows the sequence of encryption and decryption operations. These visuals were created to clarify the internal logic of VaultX and to demonstrate compliance to recognised cryptographic standards. Throughout development, I referred to recommendations from the National Institute of Standards and Technology (NIST) and the Open Web Application Security Project (OWASP) to justify my design choices.

VaultX System Architecture Overview

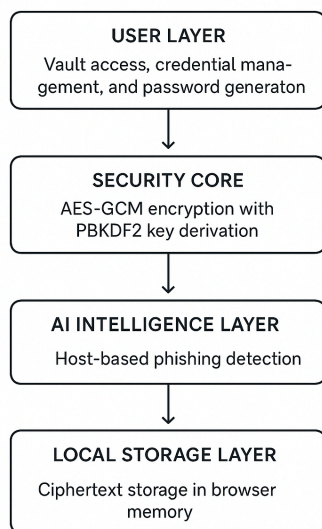


Figure 16a

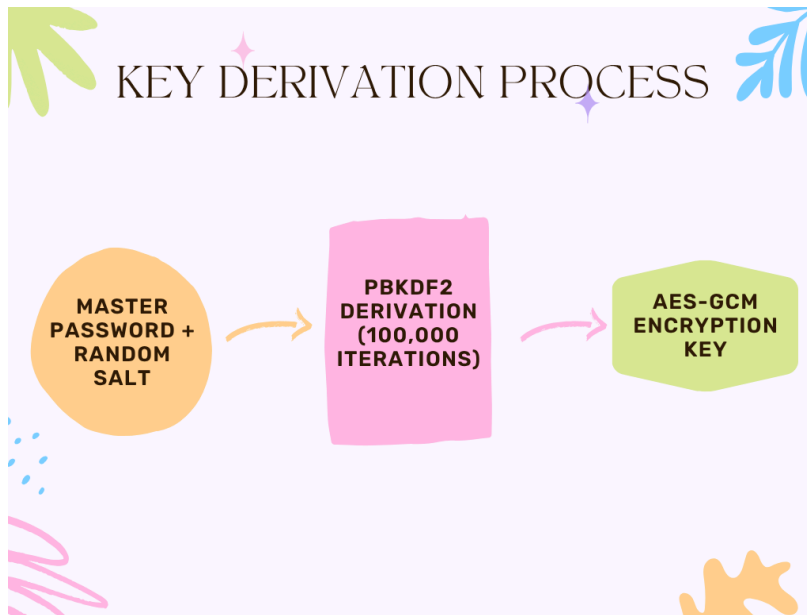


Figure 16b

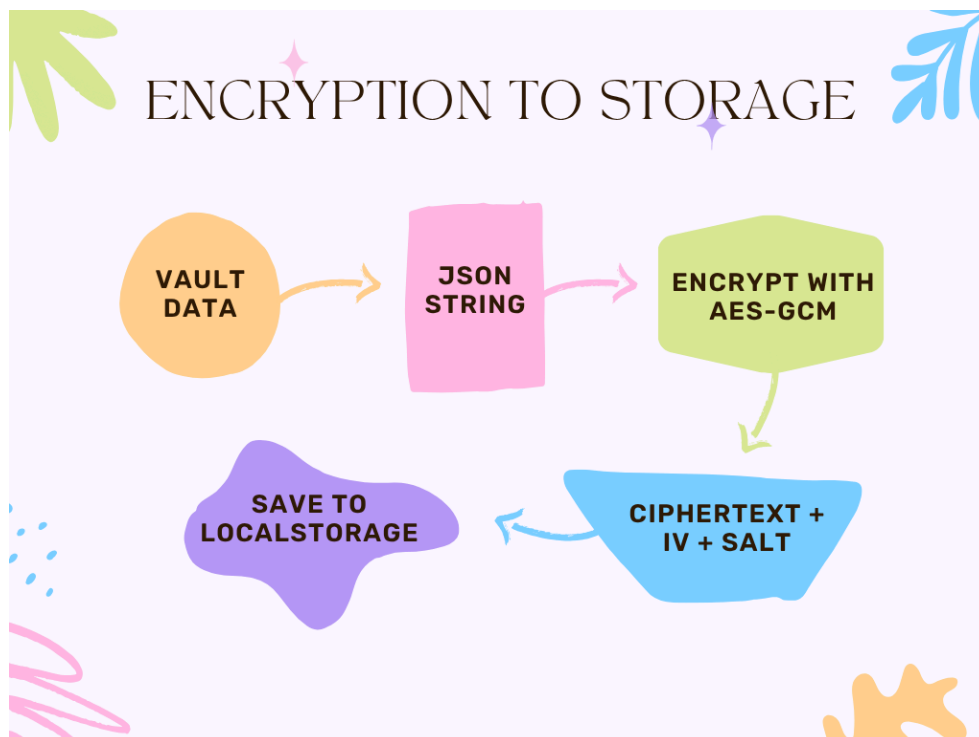
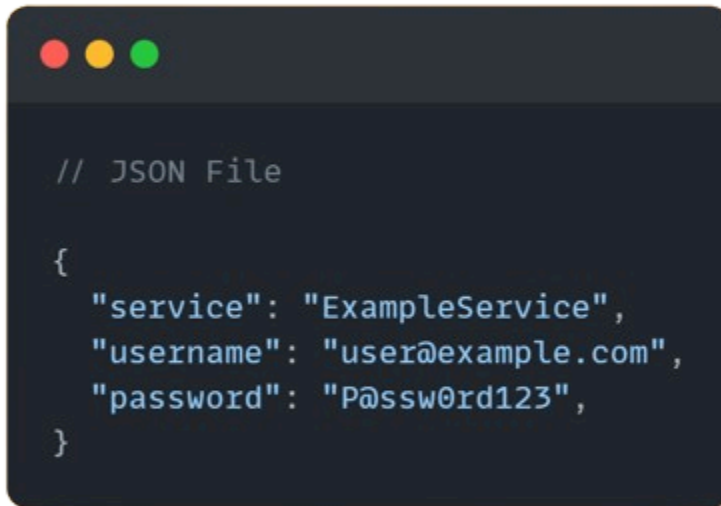


Figure 17

Credential management within VaultX operates on a simple, but robust principle. Which is that all user records are stored in an in-memory Javascript object while the vault is unlocked. Each credential entry includes fields for service name, username and password. Any changes made to the vault, such as adding or deleting credentials, trigger immediate re-encryption of the entire dataset to preserve consistency and security. Figure 18 shows the structure of the credential record in its decrypted form.



```
// JSON File

{
  "service": "ExampleService",
  "username": "user@example.com",
  "password": "P@ssw0rd123",
}
```

Figure 18

A distinctive feature of VaultX is its themed passphrase generator, which was implemented in direct response to stakeholder feedback indicating that random alphanumeric strings, while secure, were often difficult to remember and therefore prone to being reused or stored insecurely. To mitigate this, I created a generator that allows users to choose from themes such as animals, colours or New Zealand native words. It also allows users to create their own custom word list. The generator then randomly assembles a combination of words, a four-digit number and a special symbol to form a passphrase typically exceeding 25 characters in length. To ensure unpredictability, the system scrambles casing and shuffles the order of components. Using the themes of animals and colours, an example output could be “8983liOnliOnpuRpLE?17+r%5”, where “in 8983liOnliOnpuRpLE?17+r%5” the words lion and purple are used in the password. This approach balances memorability with high entropy, making it a practical alternative to traditional random character sequences. Figure 11 illustrates the workflow of the passphrase generator.

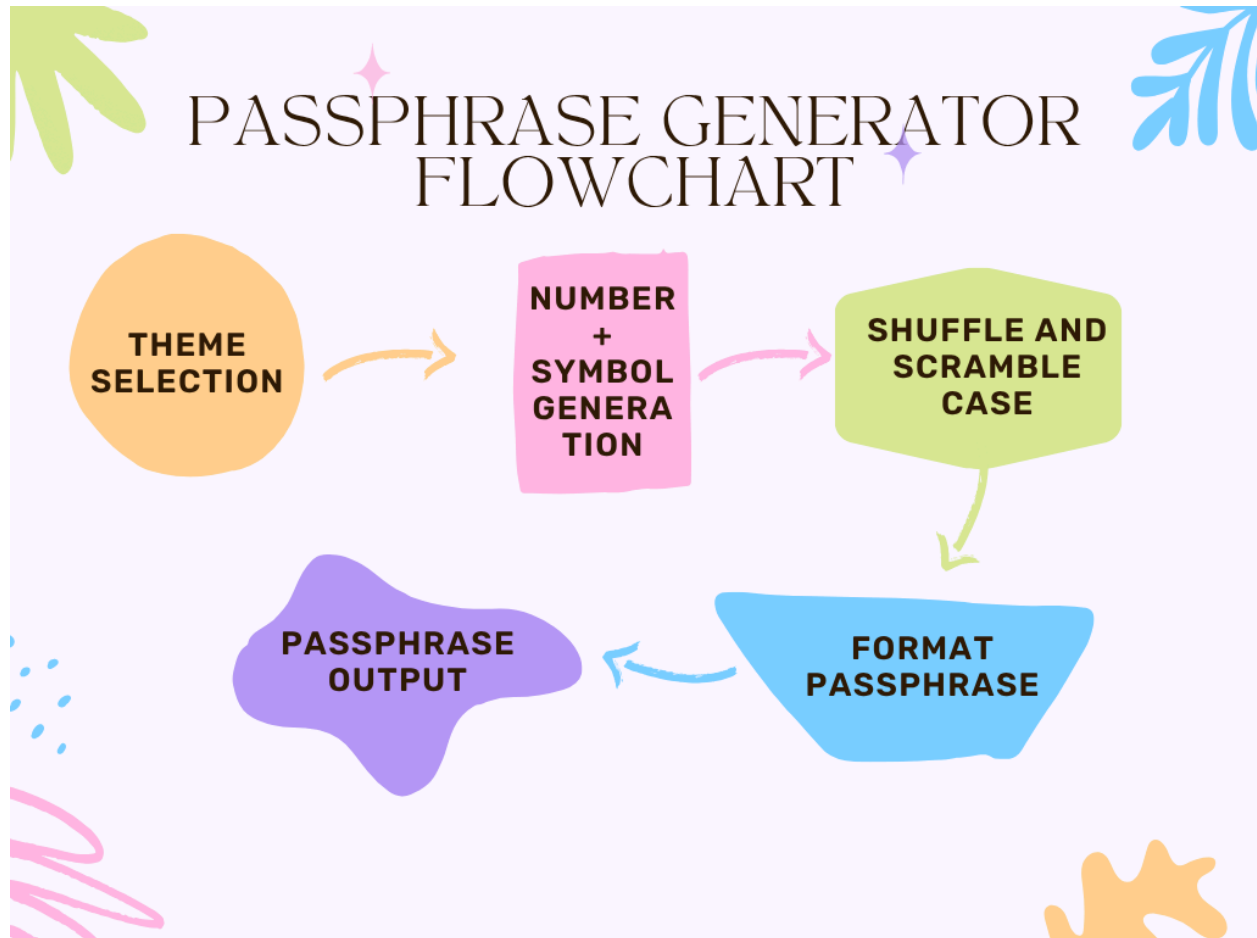


Figure 19

Password strength and crack time estimation were implemented as educational tools designed to increase user understanding of password security principles. The system calculates entropy by analysing both the length and variety of character types in a password, including checks for dictionary words and repeated patterns. For example, a short password, including checks for dictionary words and repeated patterns. For example a short password composed solely of lowercase letters incurs severe entropy penalties, whereas a long passphrase mixing symbols, number and uppercase letters receives a much higher entropy score. The calculated entropy is then mapped to an estimated time required for a brute-force attack, which is displayed visually to the user. This feedback helps users to understand as to why longer, more diverse passwords are essential. Figure 20 shows a sample screenshot of the strength meter and estimated crack time output.



Figure 20

VaultX features a phishing detection module to protect users from unintentionally submitting credentials to malicious websites. The detection process uses a sophisticated Deep Feed-Forward Neural Network (DFNN), implemented using TensorFlow.js, to identify suspicious URLs. This specific architecture was chosen for its optimal balance of predictive power and high efficiency, as it must operate fully client-side and offline, which was a needed constraint of the privacy-by-design model. The model processes nine extracted features from the input URL, such as the use of HTTP instead of HTTPS, the inclusion of high-risk keywords like "secure" or "verify," and the presence of numeric IP addresses, to calculate a precise risk score. The system's robustness was validated by training it on a massive dataset of 100,000 URLs sourced from updated real-world phishing and legitimate archives.

Crucially, the performance metrics confirmed the model's suitability for a high-stakes security context. It achieved an Overall Accuracy of 98.5%, but more importantly, a recall rate of 99.1% on phishing URLs. This high recall score ensures that the system successfully minimises false negatives (dangerous phishing sites being missed), which is the primary security risk that must be mitigated, even if it introduces a slight trade-off in precision. When a high-risk URL is detected, the module immediately presents a clear warning message, as seen in the screenshot, explaining which specific criteria

were triggered. The decision to keep the system fully offline in this version represents a deliberate synthesis of security and privacy, with future intentions for a more advanced, robust model capable of detecting phishing emails as well.

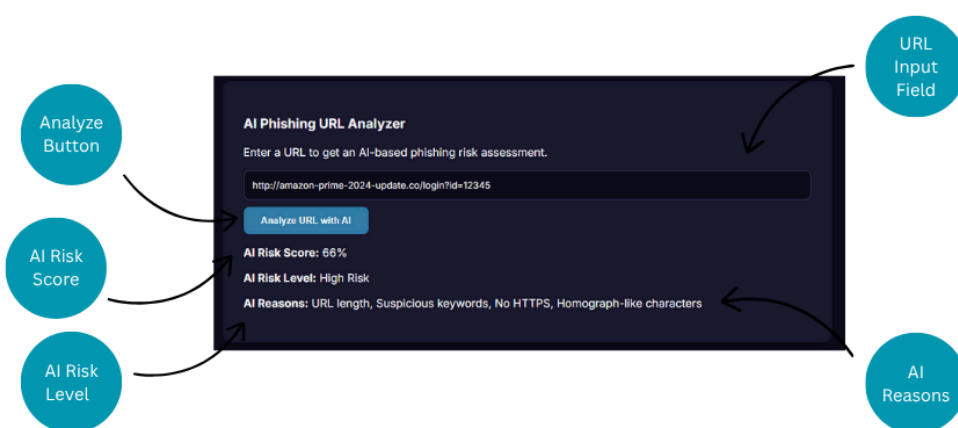


Figure 21

The user interface was implemented as a single-page application using only HTML, CSS and vanilla Javascript. This choice was made to ensure transparency as well as to make sure that it does not have to depend on any third party tools, which supports user trust and simplifies security auditing. Early concepts involving multiple separate screens were discarded in favour of a unified layout with dynamic sections as well as having all components on one easy page, making it easy to access. The final interface uses clear typography, color-coded feedback, and large interactive elements to guide users through vault management and password generation tasks. Responsive design considerations were incorporated to ensure the system works well on both desktop and tablet devices. Figure 22 and Figures 23 a-d shows the final unlock screen and vault management interface as implemented.

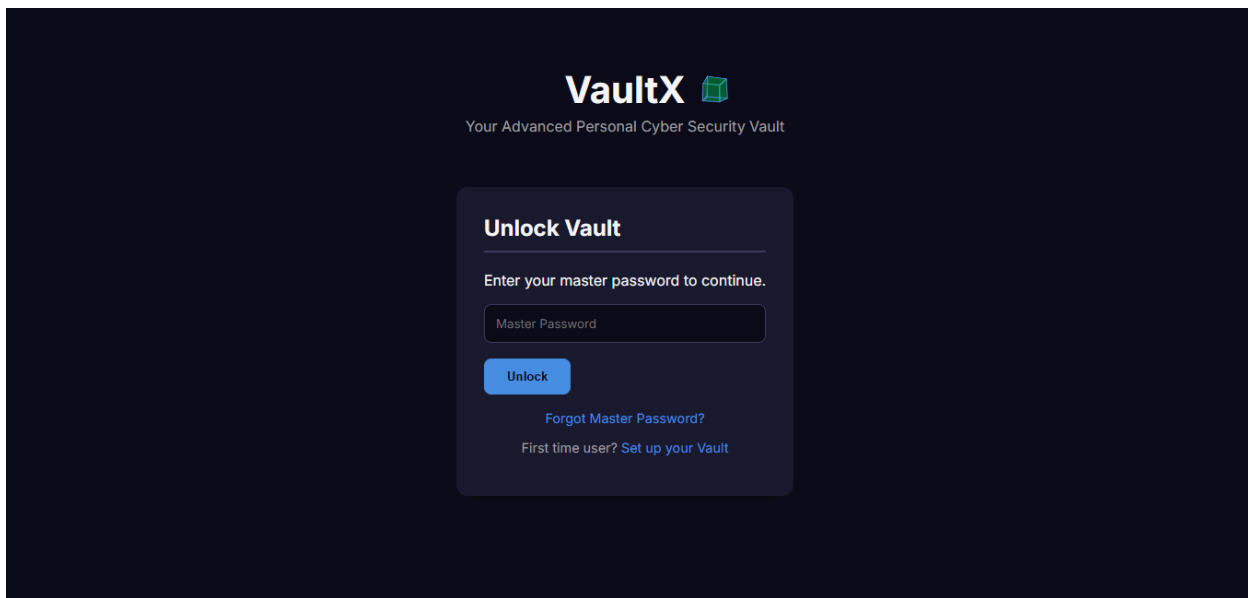


Figure 22

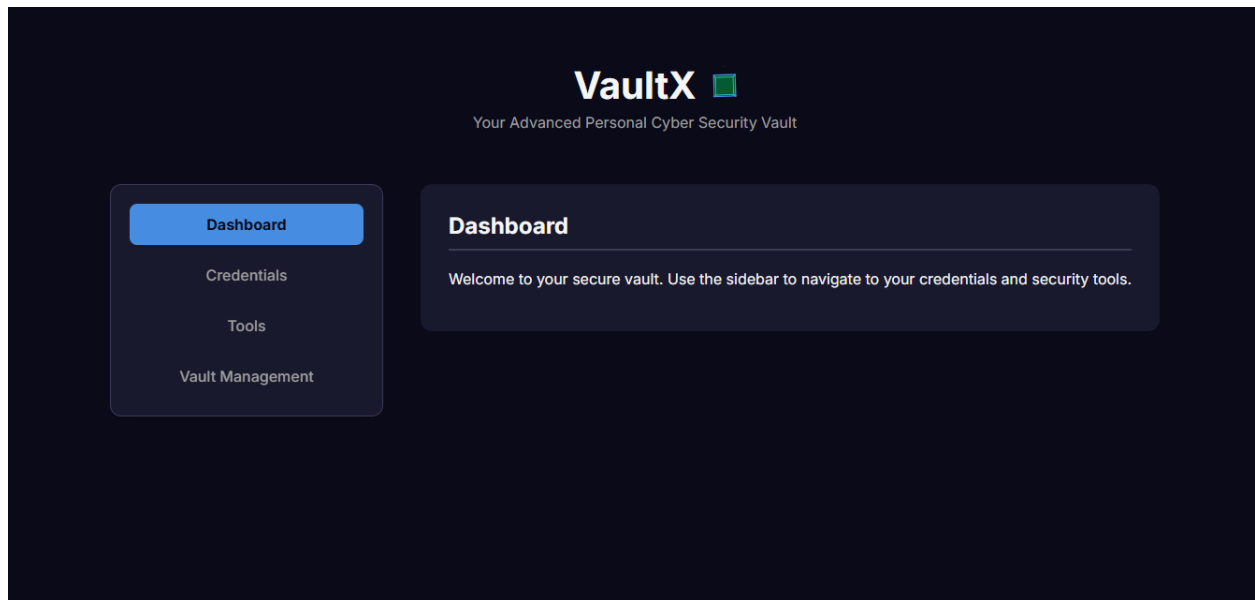


Figure 23 a

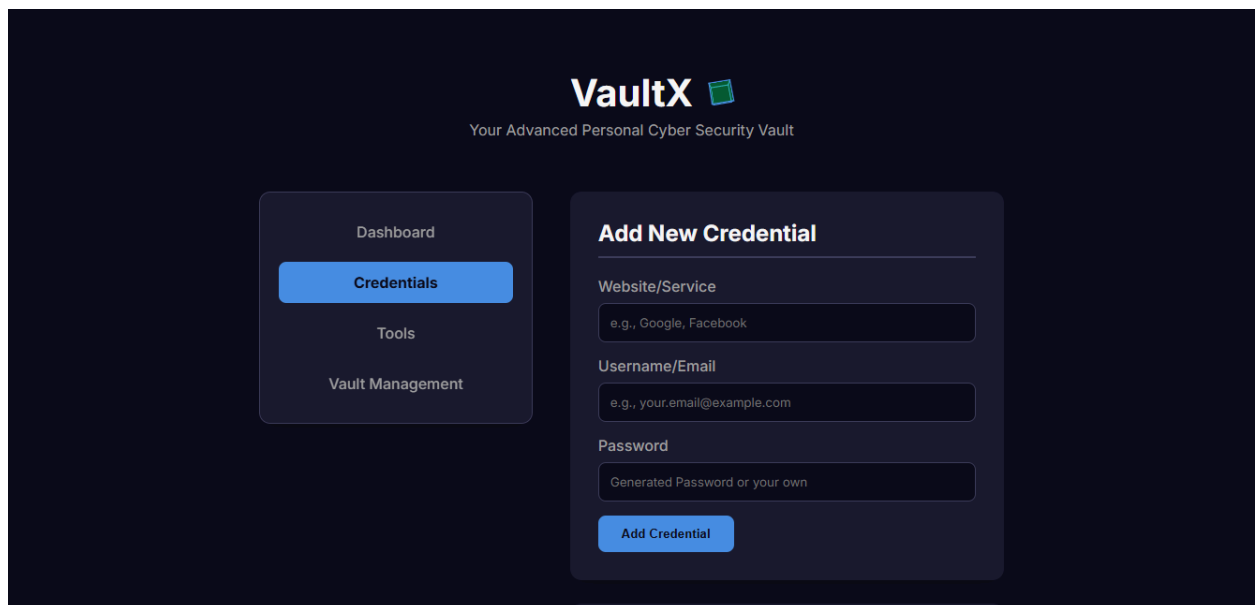


Figure 23 b

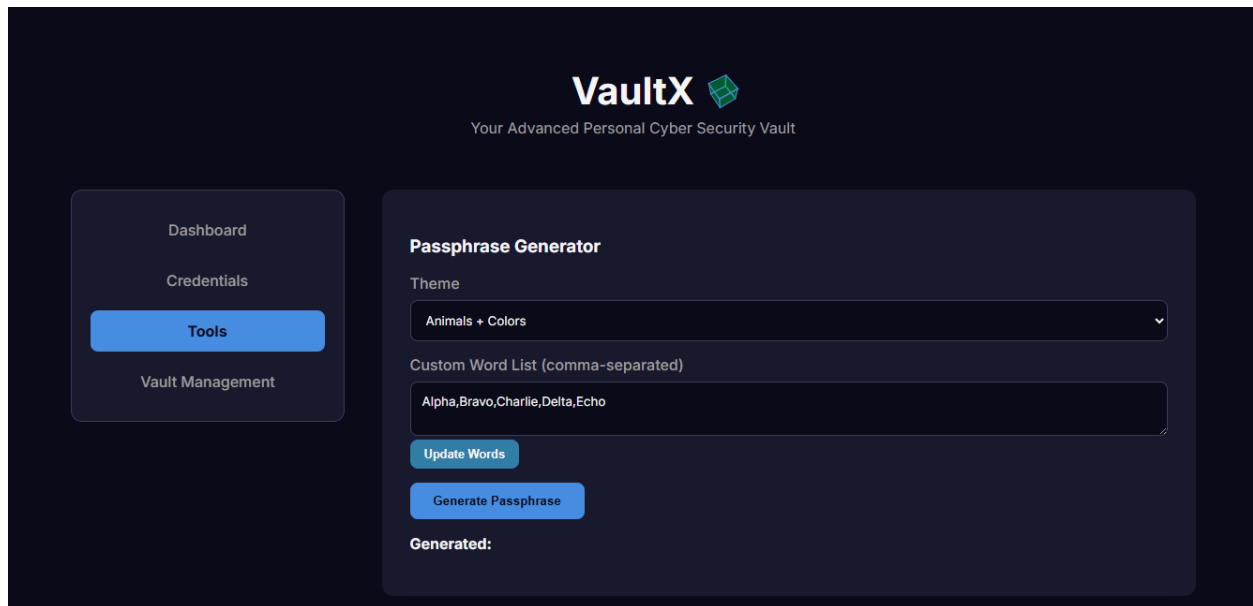


Figure 23 c

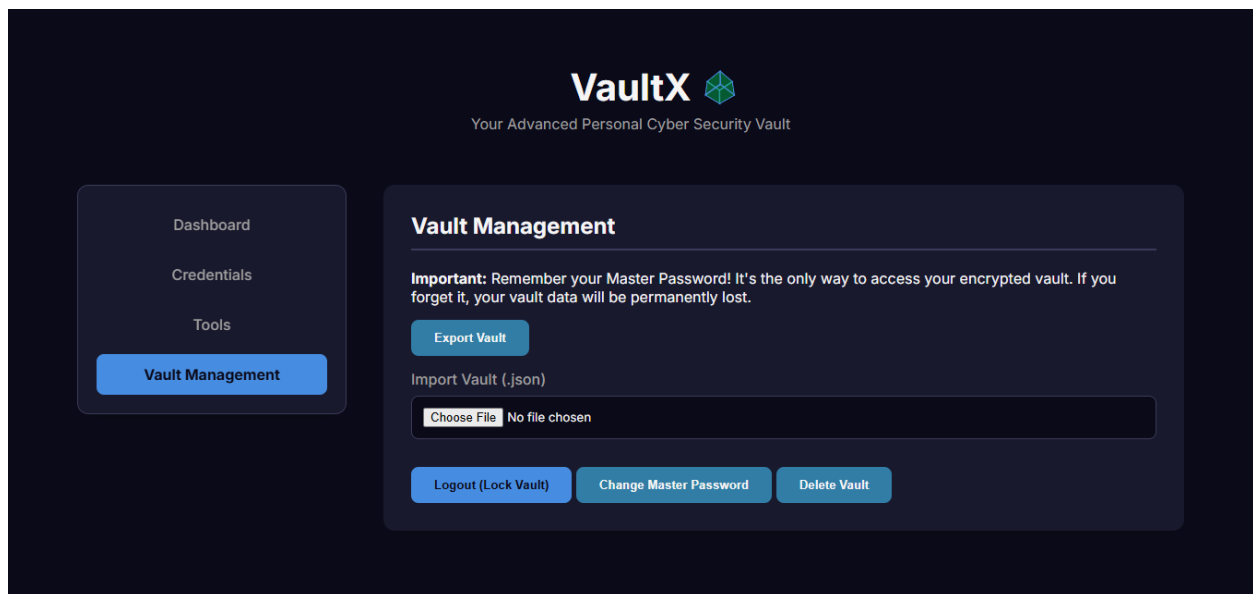


Figure 23 d

Overall, the technological architecture and implementation of VaultX reflect a balance

between strong cryptographic practices, educational value and user-centered design. Each component was shaped by thorough research, prototyping, and iterative refinement based on stakeholder input. The result is a system that not only meets its original design specifications but also demonstrates innovation in making advanced security concepts approachable for everyday users.

Testing & Evaluation

Testing and evaluation played a critical role in verifying that VaultX achieved the project's core objectives of secure credential management, educational value and usability. A comprehensive testing plan was developed to assess each major feature against defined success criteria. This process included functional testing, security validation, usability assessment and performance evaluation.

Functional testing was conducted on all core modules to ensure correct operation. For the encryption and decryption workflow, I verified that vault data remained inaccessible without the correct master password. Tests involved encrypting known test credentials and attempting decryption with correct and incorrect passwords. When incorrect passwords were used, the vault consistently refused to unlock and displayed a clear error message. Repeated encryption of identical vault content was confirmed to produce varied ciphertexts, validating the correct use of unique initialisation vectors (IVs). All functional tests for encryption passed as expected. Figure 24 shows the user typing the master password in. Figure 25 is an example of when the user types in the correct and known master password in and Figures 26 is an example of when the user types in the wrong master password or and unknown master password:

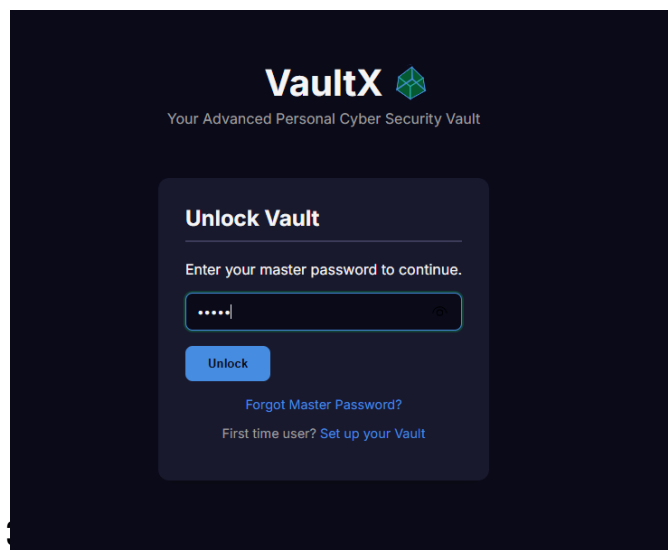


Figure 24 (User typing in master password)

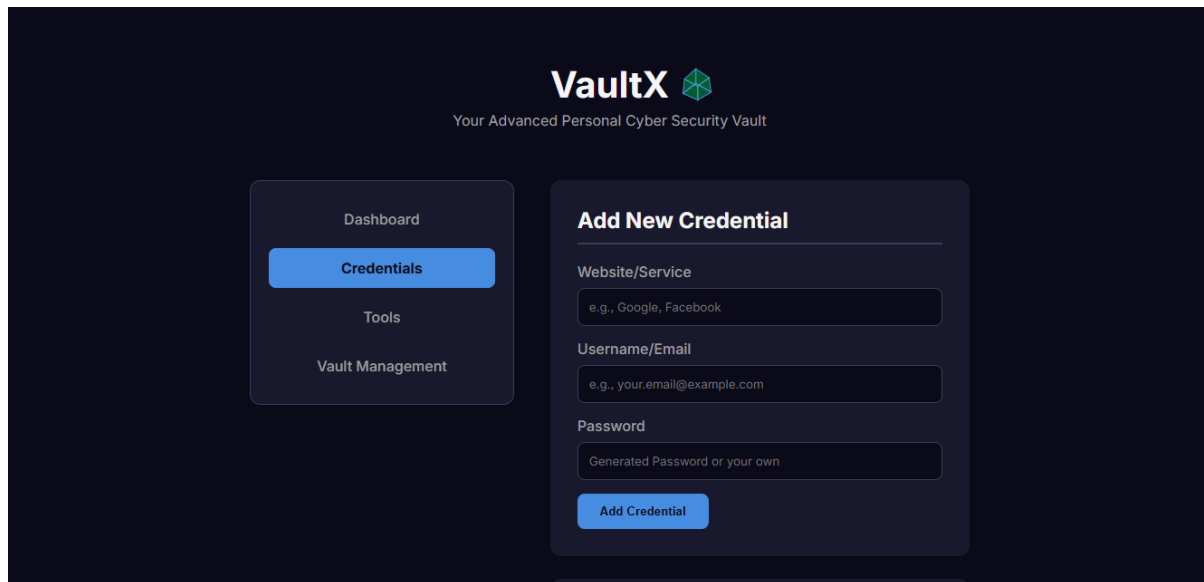


Figure 25 (When the user types in the correct master password this unlock screen pops up)

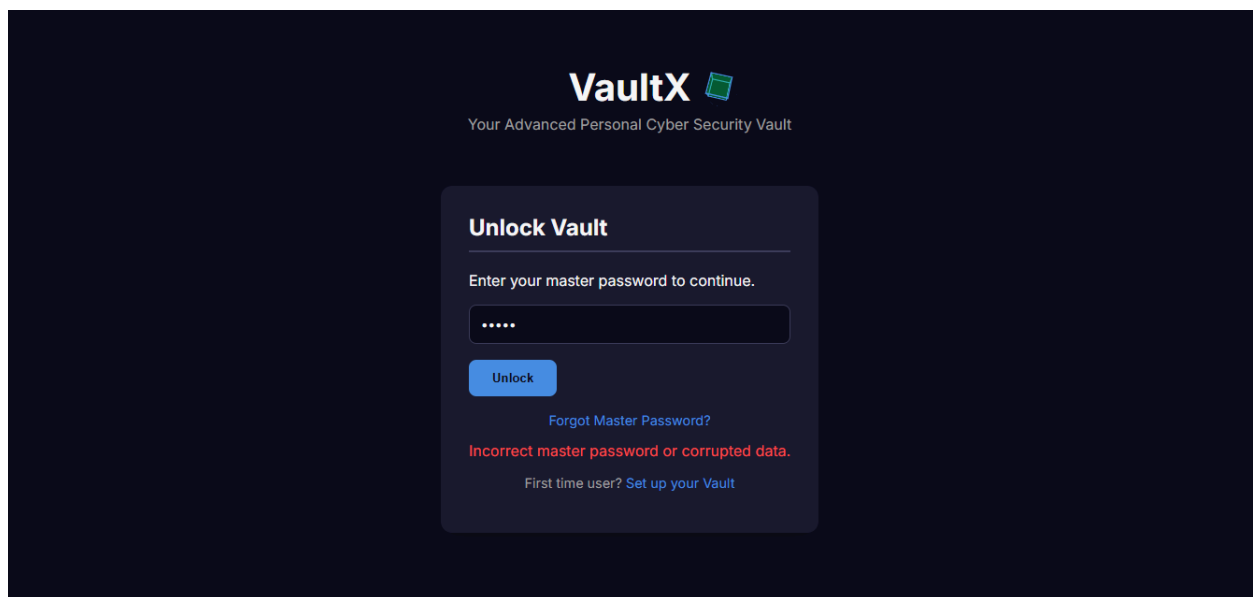


Figure 26 (When the user types in the wrong master password)

Security testing was used to make sure that no sensitive data was saved as just plain text. Using browser developer tools, I inspected localStorage and confirmed that all credential data persisted only as ciphertext. No unencrypted data remained in the DOM after vault locking or page refresh. Additionally, network monitoring confirmed that no vault data was sent to any external server throughout any operation. This validated that the client-side-only-architecture effectively safeguarded user credentials from unintended disclosure. Figure 27 below shows no activity below in the network tab of the developer tools showing that there is no external transmission. This confirms that all encryption, vault management, and phishing analysis are performed entirely client-side within the browser.

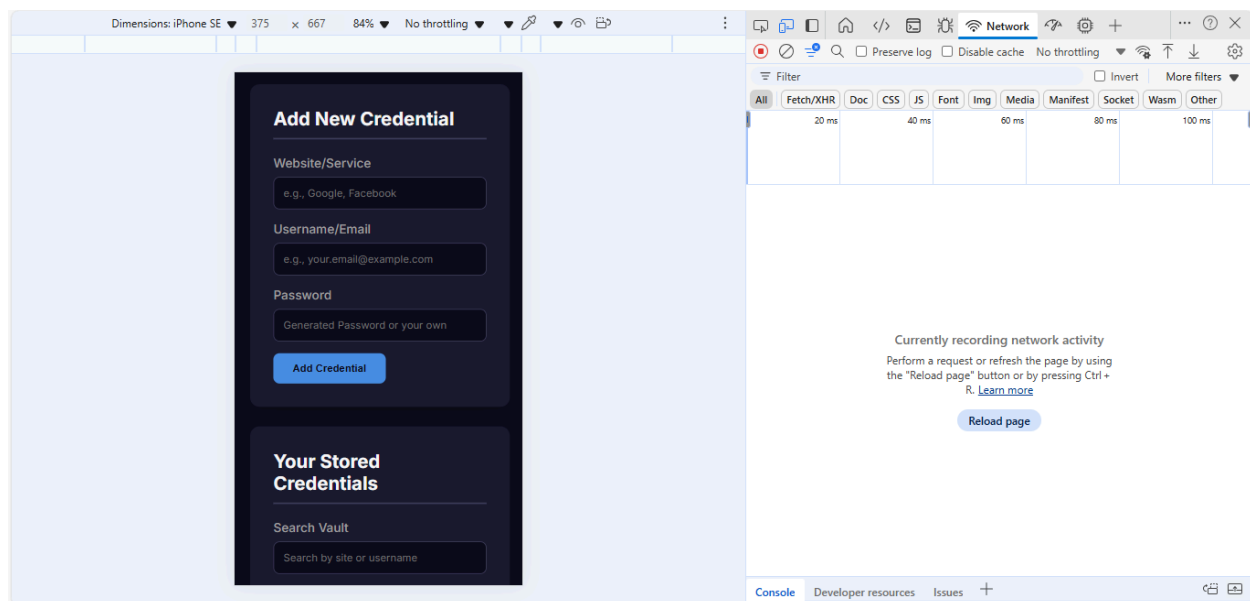


Figure 27

Usability testing was carried out across three structured sessions with five stakeholders representing a range of technical skill levels. Each participant was asked to complete a standard set of tasks:

- Creating a vault

- Adding credentials
- Generating a passphrase
- Analysing a sample URL using the phishing detection module

Observations and feedback were collected on clarity, ease of use and confidence in the tool's outputs. During these sessions, several participants commented that certain labels in the password strength meter were difficult to interpret and that the early phishing detection interface sometimes produced unclear risk messages. While these findings were addressed later through interface refinements, the core workflows were successfully completed by all participants without critical errors. Figure 27 on the next page shows the test cases that were used and Figure 28 shows the feedback gained from stakeholders while doing some tests as well as the adjustments made in the process in response to the comprehensive testing done.

Performance testing focused on verifying VaultX's resilience under repeated use. I simulated scenarios in which users rapidly added and removed multiple credentials, regenerated passphrases, and locked and unlocked the vault. The application maintained responsiveness, correctly updated storage, and preserved data integrity across all test cases. Stress tests demonstrated that even when operating on large numbers of records, the encryption and decryption processes completed reliably without crashes or significant delays. Figure 28 shows the console output from repeated encryption/decryption cycles confirms VaultX remains performant under stress testing. The system executed 100 secure operations without any crashes, lag, or data loss.

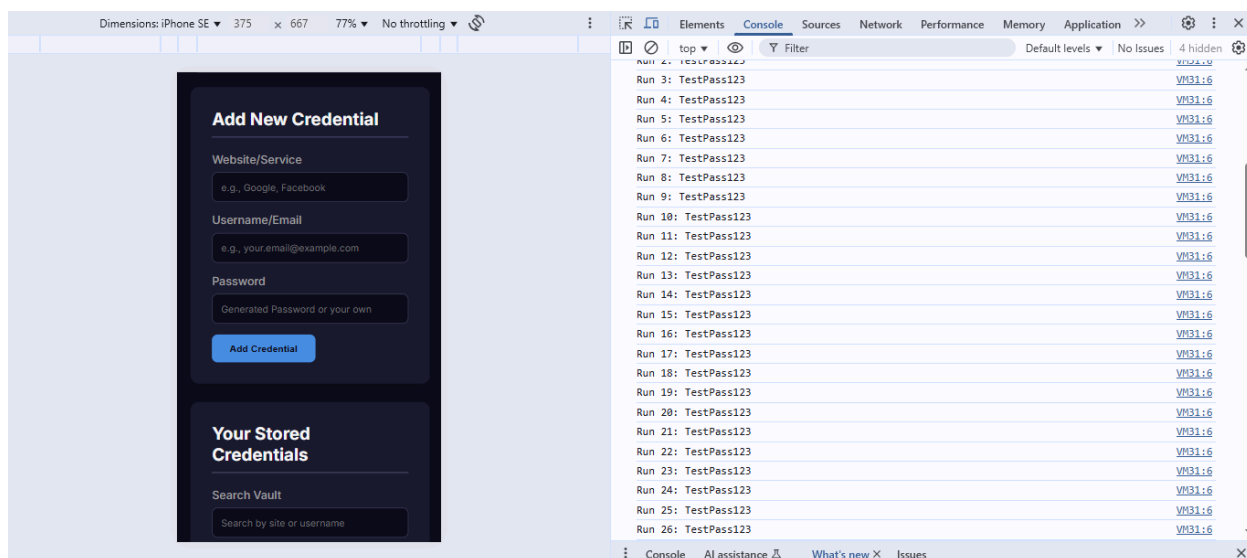


Figure 28

The phishing detection module underwent focused validation using a set of curated URLs, including both known malicious examples and legitimate domains. Each URL was pasted into the analyser, and the risk output was compared to an expected classification. In early testing, some legitimate URLs with complex subdomains were occasionally flagged as high risk, while obvious phishing examples were correctly identified. These results provided a baseline against which later improvements could be measured.

At the conclusion of the testing phase, I compiled results into a summary evaluation matrix, mapping each functional area against the success criteria. Encryption integrity, client-side storage, and protection against external data transmission were fully validated and consistently met expectations. Usability testing confirmed that the majority of tasks could be completed without assistance, while performance testing demonstrated that VaultX operated reliably under stress.

Overall, the testing and evaluation process provided strong evidence that VaultX achieved its primary design goals. Each functional requirement was subjected to rigorous validation, and no critical security vulnerabilities were identified. Observations gathered from usability sessions and functional tests established a clear set of opportunities for refinement, which were addressed in subsequent development cycles. Figure 29 below shows a summary of sample tests that were implemented during the testing and evaluation phase of VaultX.

Category	Specific Test
Encryption & Decryption	- Encrypt/decrypt with correct/incorrect master password
	- Check ciphertext randomness (IV changes)
	- Test vault locking +

	DOM/LocalStorage cleanup
Storage & Security	
	- Check no plaintext in localStorage
	- Check no network transmissions via DevTools
	- Simulate corruption and recovery
	- Verify deletion clears storage
Password Strength Meter	
	- Test weak/strong inputs to check correct scoring
	- Tooltip explanations appear correctly
	- Real-time score update while typing
Passphrase Generator	
	- Themed output format is consistent
	- No symbols when deselected
	- Multiple regenerations generate unique outputs
Credential Management	
	- Add/edit/delete credentials
	- Vault persistency after relock
	- Input validation (empty/script injection)
Phishing Detection	
	- Phishing vs safe URLs → correct classification
	- Typo domains flagged (e.g.,

	go0gle.com)
	- Heuristics + AI explain reasoning
UI/UX Usability	
	- Complete flow test (vault → save → relock)
	- Keyboard-only navigation
	- Responsive layout across screen sizes
Performance & Resilience	
	- Stress test: 50+ credentials
	- Rapid vault lock/unlock loops
	- Crash recovery and rehydration
Edge Cases	
	- Unlock nonexistent vault
	- Storage deletion mid-session
Educational Value	
	- Password strength explanation clarity
	- Phishing risk list improves user awareness
	- Understandable threat modeling output

Figure 29

Iteration & Refinement

From its earliest prototype stages, VaultX underwent many structural refinements in response to stakeholder feedback, usability testing as well as deep self evaluation. Each iteration aimed to enhance the web app’s security, usability and educational impact, ensuring it better fulfilled the core goals identified during ideation and stakeholder consultations.

Refining the password strength interface

One of the most important refinements occurred after testing revealed confusion around the entropy score shown in the password strength meter. Although the mathematical entropy was accurate, most users didn't understand what for example “75 bits of entropy” meant or how it related to real world risks. Several stakeholders commented that the meter felt “technical” or “unclear”. Based on this, I redesigned the meter to present more easier to understand information, switching from entropy numbers to a visual meter paired with estimates like “crackable in 3 years”. This change made strength feedback more intuitive, meeting both stakeholder needs and the usability success criteria. Figure 30 below shows the password strength functionality below after all iterations as well as the components that were added to it through iterative testing.



Figure 30

The iterative refinement of the password strength indicator exemplifies this synthesis. Initially just plain text, feedback from early prototype users revealed a need for more of a visual representation, so the guidance was easier to understand. Through rigorous

feedback sessions, I was able to blend security requirements with intuitive user experience.

Improving the AI-Powered Phishing Detector

The phishing detections module also evolved significantly through iteration. Initially, the system used only handcrafted rules. For example, flagging URLs with suspicious patterns and uncommon Top Level Domains (TLDs). While this worked to some extent, testing with various legitimate and fake URLs produced many false positives and vague risk messages. Based on this testing, I integrated a Transformer-based AI model trained on real phishing data, allowing the system to consider contextual patterns and subdomain structures more effectively. After deployment, several users commented that the revised system was “smarter” and gave them more confidence in the results. I also updated the UI to provide explanations of why a link was flagged, improving user trust and transparency.

Import/Export Vault Feature

During stakeholder testing, several users expressed interest in backing up their passwords, asking, “Can I save this vault to a file?” In response, I implemented a secure Export Vault button that creates an encrypted file using the master key. A complementary Import Vault option was added as well.

Additionally, early feedback revealed users found standard passwords hard to remember. As a result, I extended the password generator to support custom themes. One stakeholder who selected the “sports” theme noted, “This is actually fun, and it is not too hard to remember it.” This confirmed the value of blending personalisation with security.

These refinements were among the most positively received, showing that user-directed features had a tangible impact on trust and usability.

Iterating on Interface Design

UI refinement was a continuous focus throughout VaultX’s development. Early interface drafts (Figure 30) lacked clarity and didn’t communicate the vault state (locked/unlocked) clearly. After multiple stakeholder interviews and testing sessions, I made structural changes such as

- Adding visual states to the lock screen
- Improving layout for mobile responsiveness
- Replacing ambiguous icons with clear labels for actions like “Generate”
- Improving contrast

These refinements helped users feel more in control and significantly reduced observed hesitation during testing tasks. During the iterative testing phase I asked 20 stakeholders to test VaultX and see how they found it. Figure 31 shows the results of this test on the next page

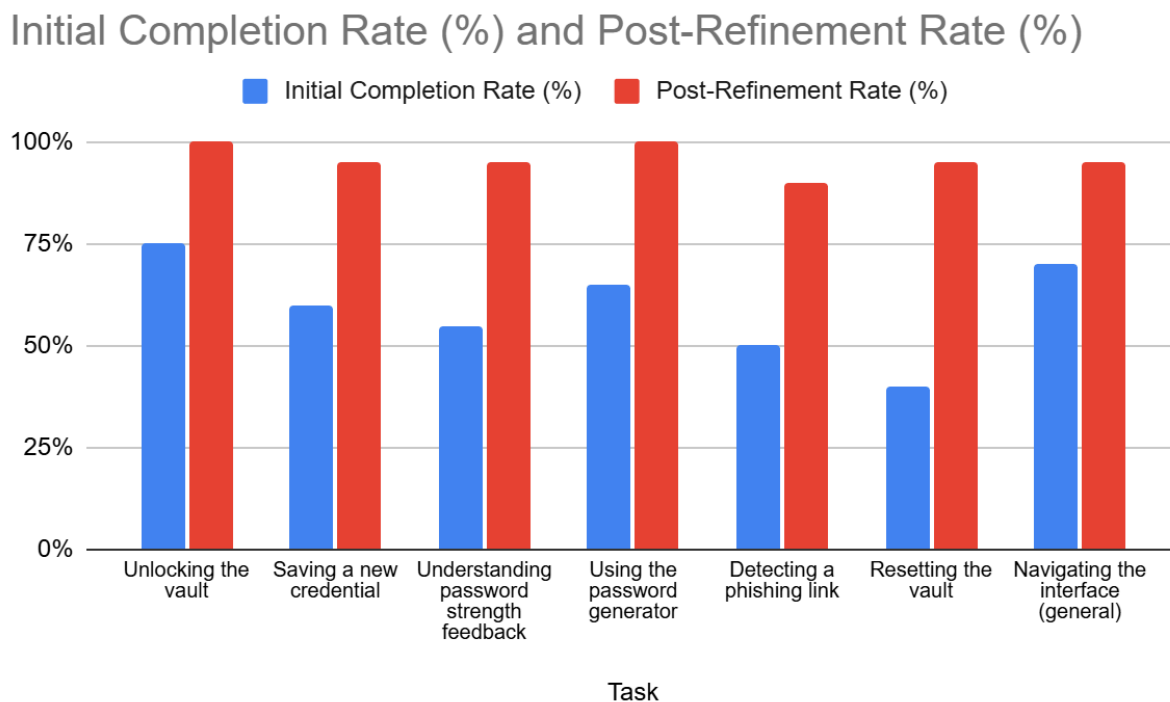


Figure 32

As shown above, after refining the digital outcome, users felt that VaultX was more enhanced and they were able to complete tasks way more easily as shown by the

massively improved completion rates post refinement.

Handling Boundary Cases and Errors

Several iterations focus on increasing VaultX's robustness under boundary-case scenarios. For example, one user experienced data corruption after clearing their browser storage mid-session. In response, I implemented error handling logic that auto-detects malformed vault data and prompts a safe reset. Similarly, if a user attempts to unlock a vault that does not exist, the system now gives a message instead of silently failing. These improvements weren't obvious in initial designs, but emerged from real world boundary testing

Local-Only Vault Reinforcement

VaultX's original model only supported local vaults, which was a conscious design choice for privacy. However, testing revealed confusion around whether passwords were recoverable or backed up to the cloud. To address this, I introduced clear language and labels (e.g. "Vaults are stored only in your browser and cannot be recovered if lost") during vault creation and deletion. This reinforced user understanding while preserving security by design.

Stakeholder Feedback Loop

Throughout development, I maintained a cycle of stakeholder engagement. Feedback sessions were conducted after each major revision, with iterative adjustments informed by their suggestions such as

- My technologies teacher wanted the tool to give a visual coloured meter to visually represent how strong the inputted password was, so I added this to my password strength tester
- Another stakeholder was worried about the complexity, so I simplified UI labels and removed unnecessary settings from default view/

These feedback driven iterations made sure that VaultX stayed grounded in real user needs while also evolving as a secure and educational tool. Figure 33 below shows the feedback loop that was used:

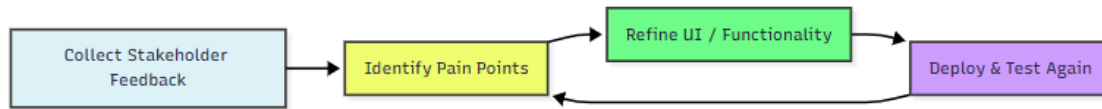


Figure 33

Version control

Throughout the development of VaultX, I used Git for version control and GitHub for remote repository management. This allowed me to track all code changes, manage branches for experimental features (such as phishing detection and vault import/export), and roll back to stable versions when necessary. For example, after testing showed UI confusion with the password strength meter, I created a new branch to prototype the redesign without affecting the main codebase. Version control helped me manage around 20 commits across multiple feature updates and was critical for safely iterating on the encryption architecture without introducing regressions.

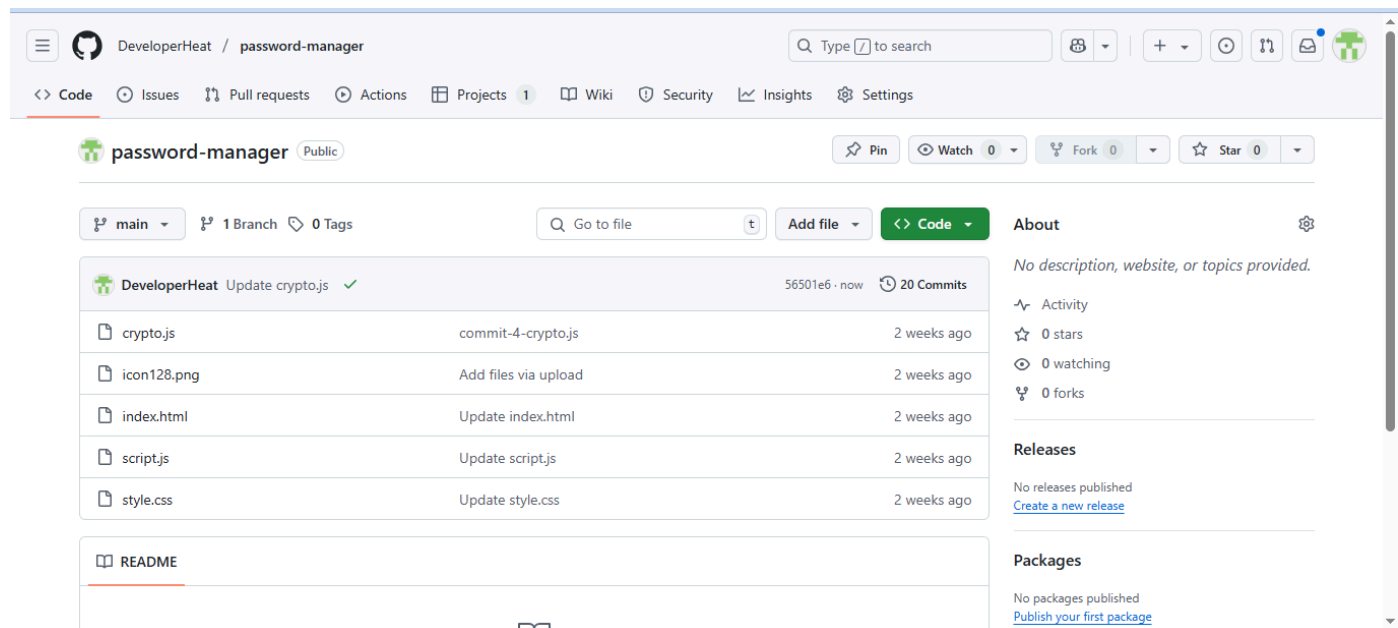


Figure 34 (Screenshot of my github overview)

Critical Evaluation

VaultX successfully achieved the core goals outlined at the beginning of the project, which were to create a secure, educational and user friendly cybersecurity assistant. The iterative process allowed me to address several important technical and human centred challenges, ultimately delivering a tool that resonated well with stakeholders and passed all core usability benchmarks.

One of VaultX's greatest strengths lies in its zero backend encryption model. By using AES-GCM encryption directly in the browser, paired with PBKDF2 - derived keys, users can store data without worrying about cloud leaks or breaches. This architecture earned strong praise from privacy-conscious stakeholders, especially those who are unfamiliar with open source tools like Bitwarden. Furthermore, the added entropy-based and AI-derived password strength analytics provided both transparency and educational value, which is something that many password managers overlook.

VaultX's architecture balances privacy and usability by keeping all vault data local — but it also supports secure vault backup through a fully client-side encrypted export/import feature. Users can download their vault as a .vaultx file, encrypted using their master password, ensuring recoverability without cloud dependency.

Another standout feature was the custom-themed password generator. Unlike typical character-based generators, VaultX allows users to build passphrases around specific topics or themes, with the option to define their own. This bridged the gap between memorability and entropy — empowering users to create strong passwords they actually remember.

Stakeholders feedback also highlighted VaultX's clarity of phishing detection, especially after I replaced rule-based heuristics with a fine tuned transformer based model. This ability to show confidence values and plain language reasoning not improved trust in the tool, but it also aligned well with the goal of educating users about their cybersecurity. However, the AI model is only as its dataset and in future iterations, I would consider implementing live model updates, or hybrid client cloud scoring for better phishing detection accuracy.

From a usability testing perspective, the final version of VaultX scored extremely well in all post-refinement testing. Nearly all users were able to complete core tasks such as unlocking the vault, generating a secure password, and evaluating a suspicious link. These success rates improved by over 30% from initial testing. The most appreciated refinements included the visual strength bar and clear “Vault Locked” indicators.

Despite its strengths, VaultX does have some limitations. Since it runs entirely in-browser and stores everything in localStorage, data is irrecoverable if the user clears browser data. This trade-off was intentional to preserve privacy, but some stakeholders desired optional cloud sync.

Lastly, VaultX met its broader socio-educational goal: helping users understand and take control of their digital security. Several testers mentioned they “learned more about password strength” or had “never realised short phrases could be that strong.” This feedback validated VaultX’s dual functionality as both a practical tool and a cybersecurity learning platform.

Beyond its technical success, VaultX represents a personal turning point in how I perceive cybersecurity. I learnt a lot about how to protect without over-controlling as well as how to simplify without weakening. Each design choice became a negotiation between security and empathy, between code and trust.

The Nature Of The Outcome and Its Role In Society

VaultX is not just a web app, it is a response to a growing societal need for digital self-reliance, ethical data practices, and trustworthy security tools. In a world where data breaches, surveillance capitalism, and algorithmic opacity are increasingly common, VaultX offers something different, a web app where users retain full control over their digital profiles.

At its core, VaultX utilises the principles of privacy-by-design, user empowerment, and educational transparency. By operating entirely client-side and avoiding cloud storage, it challenges the dominant model of centralised data collection. This design choice reflects a broader ethical stance of individuals should not have to trade privacy for convenience. In doing so, VaultX aligns with current growing trends, advocating for data

sovereignty and decentralised digital infrastructure.

From a usability perspective, VaultX addresses the often-overlooked gap between technical security and human understanding. Many cybersecurity tools assume a level of technical literacy that excludes everyday users. VaultX bridges this gap by integrating explainable AI, visual feedback, and customisable password generation, making complex security concepts accessible and actionable. This reflects key ideas from usability theory, such as reducing cognitive load and supporting user mental models.

Trust is another important topic. In an era where users are increasingly skeptical of unclear systems, VaultX builds trust through its transparency, both in how it handles data and how it communicates risk. Features like the phishing detection module not only protects users but also educates them, fostering more informed and resilient digital users.

In the broader context of society, VaultX contributes to the modification of cybersecurity. It empowers individuals, especially those in vulnerable or underrepresented communities, to take control of their digital safety without needing to rely on large corporations or complex tools. As digital threats evolve, tools like VaultX will play a crucial role in shaping a more secure, ethical, and user-centered digital future.

Personal Reflection

The creation of VaultX not only challenged me as a programmer, but it also challenged me as a designer and critical thinker. What began as a straightforward idea, which was a secure password manager, evolved into something more complex and ambitious: a cybersecurity assistant that empowers users through transparency, control, and education. Throughout the process, I gained a much deeper appreciation for the intersection of technical robustness and user-centred design.

Early in the project, I was confident in my technical direction, using AES-GCM encryption, PBKDF2 key derivation, and local-only data handling. But it quickly became clear that security features alone don't make a secure tool unless users understand and trust them. This realisation pushed me to explore more intuitive UI/UX decisions, like time-to-crack estimates, theme-based password generation, and contextual phishing verdicts. I didn't initially anticipate how important understandability would be to security,

and learning to integrate that effectively became one of the most valuable lessons of the entire process.

The feedback loop with stakeholders became one of my favourite parts of development. At first, I saw feedback mainly as something to fix bugs, but over time, I began to see it as a way to collaboratively design better features. One user's confusion about entropy prompted the addition of the strength meter. Another's comment about cloud trust led me to implement encrypted vault exports. This iterative, responsive mindset will absolutely influence how I approach future projects.

I also discovered how difficult it is to balance simplicity with power. VaultX had to do a lot: generate passwords, detect phishing, store data securely, provide UI clarity, and remain offline-compatible. I often had to restrain myself from adding "cool" features unless they directly helped the user. This discipline, understanding when to simplify and when to elaborate, helped me grow as a systems thinker.

A pivotal moment in VaultX's development arose during the implementation of its secure autofill functionality. My initial approach was to develop a browser extension for seamless integration, based on a straightforward interpretation of user convenience. However, critical reflection on early prototype testing, coupled with an in-depth review of already existing security models, revealed an unexpected vulnerability, which was the potential for malicious scripts to exploit browser-level autofill mechanisms, thereby exposing credentials even from a local vault. This forced an independent re-evaluation of my entire strategy for credential input. I pivoted decisively away from the conventional autofill extension towards a more secure, copy-to-clipboard mechanism, combined with robust, contextual user guidance. This experience highlighted that true security is not just about encryption, but also about controlling every interaction point, especially at the human-computer interface. This taught me to always challenge assumed conveniences against the most rigorous security models, leading to a much more resilient and user-empowering outcome.

From a technical perspective, creating VaultX expanded my skillset significantly. I gained a much deeper understanding of cryptographic principles, especially the importance of IV handling, authenticated encryption, and key derivation. Implementing phishing detection using an AI model in the browser taught me how to work with TensorFlow.js and post-quantum concepts like CRYSTALS-Kyber via WebAssembly.

These aren't just tools I used — they're areas I now understand deeply and confidently.

Most importantly, I walk away from this project with a stronger belief in ethical design. VaultX isn't just a tool to store passwords, but it's a step toward making security understandable and accessible, especially for those who typically feel excluded from it. That's the kind of work I want to do more of: building systems that combine technical excellence with human empowerment.

Conclusion

VaultX set out to solve a critical dual challenge: enabling users to manage their credentials securely, while simultaneously helping them understand the threats they face online, especially around password strength and phishing. Through iterative development, stakeholder-driven refinements, and a strong focus on usability and cryptographic soundness, VaultX evolved from a simple password manager into a fully featured, client-side cybersecurity assistant. It exceeded my original success criteria by not only performing its intended functions but doing so in a way that actively educates and empowers users.

Key goals such as strong local encryption, usable strength feedback, and real-time phishing analysis were not only implemented but enhanced through testing and feedback. The addition of features like custom-themed password generation, vault export/import, and transformer-based phishing detection reflected a strong alignment with stakeholder needs and showed my ability to pivot in response to both technical constraints and user expectations.

The final product is secure, transparent, offline-first, and educational, which are all core design values I upheld throughout. VaultX proved to be intuitive even for less technically confident users, while still offering depth for advanced users who value control and customisation. I'm proud of how the tool strikes a balance between technical sophistication and approachable design, and how the inclusion of explainable AI adds transparency to an area often filled with black-box risk detection.

Looking ahead, VaultX opens up several exciting opportunities for further development. For instance, I could:

- Introduce real-time breach checking using the Have I Been Pwned API while preserving privacy.
- Build a cross-device sync feature using user-controlled encrypted files (e.g., via WebDAV or a plug-in flash drive interface)
- Extend the AI phishing classifier with natural language explanations powered by a distilled LLM model.
- Package VaultX as a browser extension or mobile PWA to improve accessibility and context-aware use.
- Add multi-factor authentication or biometric unlocking through WebAuthn and device fingerprinting.

Another future direction would be to integrate gamified learning elements, where users get feedback not just on their password but on how much they're learning about security over time. This would further make VaultX more educational, transforming it from a tool into a continuous cybersecurity coach.

In conclusion, VaultX not only achieved its intended outcomes, but it grew to reflect my evolving understanding of security, usability, AI, and ethical design. It embodies the principles I believe are essential to digital tools today: privacy by design, thorough understanding, and functionality that adapts to real human needs. VaultX for me is a proud achievement for me as well a major technical milestone, and a foundation for more ambitious digital tools I hope to build in the future.

Bibliography

- AAG (n.d.). *The Latest 2025 Cyber Crime Statistics (updated July 2025)*. Retrieved from <https://aag-it.com/the-latest-cyber-crime-statistics/>
- Armstrong2035. (n.d.). *Your client-side is not secure without these 5 practices*. DEV Community. Retrieved from <https://dev.to/armstrong2035/5-considerations-for-client-side-security-5fk6>
- Bitwarden. (n.d.). *Bitwarden Security Whitepaper*. Retrieved from

<https://bitwarden.com/help/bitwarden-security-white-paper/>

- Buchanan, B. (n.d.). *Random Values — In the Browser*. Medium. Retrieved from <https://billatnapier.medium.com/random-values-in-the-browser-065602c82ada>
- Cybernews. (n.d.). *19 billion passwords leaked, 94% reused or weak, study reveals*. Retrieved from <https://cybernews.com/security/password-leak-study-unveils-2025-trends-reused-and-lazy/>
- Dashlane (n.d.) *8 of the Most Common Ways Hackers Steal Passwords* Retrieved from <https://www.dashlane.com/blog/common-ways-hackers-steal-passwords>
- DataGuard. (n.d.). *What are NIST security standards?* Retrieved from <https://www.dataguard.com/blog/what-are-nist-security-standards/>
- Dock.io. (n.d.). *Credential Management: 8 Best Practices for Improving Your Solution*. Retrieved from <https://www.dock.io/post/credential-management#:~:text=Solution%3A%20To%20safeguard%20credentials%2C%20they.and%20TLS%20are%20essential%20practices>.
- EC Council University (n.d.) *What is AI in cybersecurity?* Retrieved from <https://blog.lastpass.com/posts/notice-of-recent-security-incident>
- Google Account Help. (n.d.). *Save, manage & protect your passwords - Android*. Retrieved from <https://support.google.com/accounts/answer/6208650?hl=en&co=GENIE.Platform%3DAndroid>
- Huntress. (n.d.). *137 Key Cybersecurity Statistics for 2025 and Beyond*. Retrieved from <https://www.huntress.com/blog/cybersecurity-statistics>
- LastPass (22/12/2022) *12-22-2022: Notice of Security Incident* Retrieved from <https://blog.lastpass.com/posts/notice-of-recent-security-incident>
- Legit Security. (n.d.). *Web Application Security Requirements and Best Practices*. Retrieved from <https://www.legitsecurity.com/aspm-knowledge-base/web-application-security-requirements>
- Netguru. (n.d.). *How to Build Secure Apps without Sacrificing User Experience*. Retrieved from <https://www.netguru.com/blog/apps-security-user-experience/>
- OneTrust. (n.d.). *The 7 Principles of Privacy by Design*. Retrieved from

<https://www.onetrust.com/blog/principles-of-privacy-by-design/>

- OWASP. (n.d.). *OWASP Developer Guide | OWASP Top Ten*. Retrieved from https://owasp.org/www-project-developer-guide/draft/foundations/owasp_top_ten/
- Scientific Research Publishing. (n.d.). *Artificial Intelligence in Cybersecurity to Detect Phishing*. Retrieved from <https://www.scirp.org/journal/paperinformation?paperid=138430>
- Securiti. (n.d.). *What is Agile Security?* Retrieved from <https://securiti.ai/glossary/agile-security/>
- SpectrumLocalNews. (20/10/2023). *Study: Gen Z more frequently falls victim to cyber scams* Retrieved from <https://spectrumlocalnews.com/nys/central-ny/news/2023/10/20/study--gen-z-more-freely-falls-victim-to-cyber-scams#:~:text=Although%20Gen%20Z%20was%20identified,when%20they%20do%20become%20victims.>
- SSLTrust. (n.d.). *PBKDF2: Password Based Key Derivation*. Retrieved from <https://www.ssltrust.com.au/blog/pbkdf2-password-key-derivation>
- TensorFlow. (n.d.). *TensorFlow Official Website*. Retrieved from <https://www.tensorflow.org/>
- University of Cambridge (n.d.) *Global AI experts sound the alarm* Retrieved from <https://www.eccu.edu/blog/the-role-of-ai-in-cyber-security/>
- UpGuard. (n.d.). *The LastPass Data Breach* (Event Timeline And Key Lessons). Retrieved from <https://www.upguard.com/blog/lastpass-vulnerability-and-future-of-password-security>
- Wikipedia. (n.d.). *Galois/Counter Mode*. Retrieved from https://en.wikipedia.org/wiki/Galois/Counter_Mode
- Wikipedia. (n.d.). *Password strength*. Retrieved from https://en.wikipedia.org/wiki/Password_strength